2021 IEEE Latin American Conference on Computational Intelligence (LA-CCI) | 978-1-7281-8864-5/21/$31.00 ©2021 IEEE | DOI: 10.1109/LA-CCI48322.2021.9769844

Towards best default configuration settings for NMPSO in multi-objective optimization

1st Rodrigo Marinao-Rivas
Department of Civil Engineering
Universidad de La Frontera
Temuco, Chile
r.marinao01@ufromail.cl

2nd Mauricio Zambrano-Bigiarini
Department of Civil Engineering
Universidad de La Frontera
Temuco, Chile
https://orcid.org/0000-0002-9536-643X

***Abstract*—In this work we tested different configuration settings for the NMPSO algorithm, aiming at solving multi-objective optimization problems with a small number of function evaluations, which is an important aspect that must be addressed in real-world optimization problems. Sixteen different configurations were tested for NMPSO, with different combinations of: i) the swarm size, ii) the maximum number of particles in the external archive, and iii) the maximum amount of genetic operations in the external archive. Three DTLZ problems were used to select the best configuration, which was then evaluated against other state-of-the-art multi-objective optimization algorithms (MMOPSO, NSGA-II, NSGA-III). Our results showed that the fastest convergence towards the true Pareto-optimal front is provided by the configuration with a swarm size of 10, a maximum number of particles allowed in the external archive of 100, and a limit of genetic operations per iteration given by 50% of the maximum number of particles allowed in the external archive. The selected configuration was also very competitive or even superior against NSGA-II and NSGA-III, in terms of the number of function evaluations required to start having an HV larger than zero, but also in the HV values achieved after stabilization of the Pareto-optimal front.**

***Index Terms*—particle swarm optimization (PSO), multi-objective optimization, Pareto-optimal front (POF), NMPSO**

I. Introduction

Several engineering applications usually face problems where competing objectives needs to be optimized at the same time [1]–[6], thus forming what is known as multi-objective optimization problems (MOPs) [7]–[9]. The way to address these problems is not focused on finding a single solution that simultaneously optimizes all the objectives, as instead it is assumed that improvement in one objective can only be obtained by deteriorating one or more of the remaining objectives, which leads to considering several possible solutions. For a set of solutions to be considered optimal in a MOP, each solution must be non-dominated, which implies that none of the remaining solutions is better for all the proposed objectives [10]. The boundary defined by all the non-dominated solutions forms a Pareto-optimal front (POF), which represents the best compromise among the objectives [11]. The POF is a central concept on which most of the multi-objective optimization techniques are based, so in solving problems, the solutions involved are usually stored in a repository [12], [13], also called external archive (A).

In the last decades, several optimization techniques have been developed to find Pareto-optimal front for different MOPs, being multi-objective evolutionary algorithms (MOEAs) [14]–[16] and multi-objective particle swarm optimizers (MOPSOs) [13], [17]–[20] two of the biggest nature-inspired families of heuristic algorithms. MOPSOs use a population of *particles* within a decision variable space, to find the POF in the objective space. Some current MOPSOs are OMOPSO [17], CSMPSO [18], SMPSO [19], and CMPSO [20]. In recent years, [21] introduced NMPSO, a novel MOPSO that features a balanceable fitness estimation (BFE) method to maintain diversity within the POF. The BFE method accelerates the convergence of candidate solutions to the true POF by combining the information of convergence and diversity for each individual. In addition, NMPSO had: i) a novel velocity update equation, where an extra search direction was introduced, pointing from the personal best particle to the global best ones; ii) an evolutionary search on the external archive, where simulated binary crossover (SBX) and polynomial-based mutation (PBM) are used to generate the non-dominated solutions, and iii) a new selection mechanism for the archive update, where the BFE is used after the PSO search to select the non-dominated solutions that will be kept in the external archive, avoiding the unnecessary growth of its size during iterations.

Although NMPSO has shown to be highly competitive in solving many-objective optimization problems (MaOPs, i.e., problems with more than three objectives) [21], we have not found in the literature an algorithm configuration that makes it competitive in solving MOPs (i.e., problems with three objectives or less), which are typical of many real-world engineering applications. Therefore, in this work we explore different configuration settings for NMPSO in order to provide guidance for its implementation in multi-objective problems to be solved with a reduced number of function evaluations. In particular we analyze the following three parameter settings of NMPSO: i) the size of the swarm used in the PSO search, ii) the maximum number of particles in the external archive, and iii) the maximum amount of genetic operations in the external archive that can undergo crossover and mutation. The

This work was partially funded by ANID PCI NSFC 190018, ANID Fondecyt 1212071.

configuration we are looking for puts special emphasis on the number of function evaluations, because the execution time of the functions to be optimized in real-world optimization problems (e.g., numerical models) can be very long, resulting in an important restriction to be addressed.

The remainder of this article is organized as follows: Section II provides the basics on the original NMPSO algorithm, and some modifications carried out to improve its performance with MOPs. Section III describes the experimental setup used to identify optimal settings of NMPSO for real-world applications. Results are presented in Section IV, and Section V concludes and summarizes the main findings of this work.

II. NMPSO IMPLEMENTATION

A. Original NMPSO algorithm

NMPSO [21] is a novel MOPSO that combines a PSO-based mechanism and evolutionary mechanism [22] to maintain diversity and accelerate convergence to the Pareto-optimal front. A balanceable fitness estimation (BFE) method is used to rank particles in an *external archive* (A), in order to provide an effective guidance to the true POF, while keeping diversity among particles.

Similar to the canonical PSO algorithm [23], NMPSO starts with a random initialization of the particles' positions within the decision variable space. Considering a D-dimensional search space, the position for the i-th particle is represented by $\vec{X}_i = x_{i1}, x_{i2}, ..., x_{iD}$. The performance of each particle is evaluated through the specific value of each objective function, which is the basis for updating $\vec{X}_i$. The personal/previous best, i.e., the best-known position of the i-th particle, is represented by $\vec{P}_i = p_{i1}, p_{i2}, ..., p_{iD}$, whereas the global best, is represented by $\vec{G}_i = g_{i1}, g_{i2}, ..., g_{iD}$. In NMPSO, the global best of each particle ($\vec{G}_i$) is randomly selected from the 10% particles with better BFE values in the external archive [21].

For the PSO search, NMPSO integrates a new formulation for velocity update equation [21], as follows:

$$\begin{aligned}\vec{V}_i^{(t+1)} = \omega\vec{V}_i^t + c_1\vec{U}_1^t \otimes (\vec{P}_i^t - \vec{X}_i^t) \\ +c_2\vec{U}_2^t \otimes (\vec{G}^t - \vec{X}_i^t) \\ +c_3\vec{U}_3^t \otimes (\vec{G}^t - \vec{P}_i^t)\end{aligned} \quad (1)$$

where $i = 1, 2, ..., N$, with N equal to swarm size; and $t = 1, 2, ..., T$, with T equal to the maximum number of iterations. ω is the inertia weight, c_1 and c_2 are the cognitive and social acceleration coefficients, and $\vec{U}_1$ and $\vec{U}_2$ are independent and uniformly distributed random vectors in $[0, 1]$ (note that $\otimes$ denotes element-wise vector multiplication). c_3 and $\vec{U}_3$ are the coefficient and uniformly distributed random vector, respectively, for the last term in the velocity equation. This last term has been reported to contribute to finding a better POF for MaOPs [21]. However, in preliminary tests (not shown here), for MOPs (up to three objective functions) it introduces an important burden in terms of the number of model evaluations required to achieve the POF. Therefore, in the tests carried out in this work, this term has not been considered.

Then, the position of the i-th particle is updated as follows:

$$\vec{X}_i^{(t+1)} = \vec{X}_i^{(t)} + \vec{V}_i^{(t+1)} \quad (2)$$

After the PSO search, the non-dominated particles in the swarm are added to A to keep track of the POF along the iterations. When the size of the updated repository is larger than the maximum size of the external archive (N_e) the BFE method is used to select non-dominated solutions to be kept in A, which are considered to be good representatives of the entire POF. Afterwards, NMPSO performs evolutionary operations on non-dominated solutions in A, to allow beneficial exchange of information among its particles. Specifically, the simulated binary crossover (SBX) operator is used to allow the interchange of useful gene segments, and polynomial-based mutation (PM) is carried out to add a small disturbance to improve local search. Therefore, if the size of the current external archive is N_A, with $N_A \leq N_e$, the two previous evolutionary operations impose a number N_A of additional evaluations for each iteration. More details on BFE and the two genetic operations can be found in [21] and [22], respectively.

B. NMPSO configuration settings

In order to provide some guidance on the best configuration settings for using NMPSO in real-world MOPs, in this article we analyze three basic parameters of this algorithm. These three configuration settings are briefly explained in the following lines.

1) Swarm size (N): The PSO search is carried out by the N particles in the swarm, following the velocity and position update equations presented in (1) and (2), respectively. In NMPSO the position of the N particles is randomly initialized in the D-dimensional search space, while the velocities are set to zero ($v_i = 0$).

2) Maximum size of the external archive (Ne): The number of particles in the external archive (N_e) impose an upper boundary for the number of model evaluations to be run in each iteration on top of those already carried out during the PSO search (see Section II-B3). Therefore, it is important to limit the size of A to prevent an uncontrolled increase in the number of model evaluations required for NMPSO. However, [21] only mentions that A starts empty, without a clear recommendation for the maximum number of particles on it (N_e).

3) Maximum amount of genetic operations ($MaxGO$): According to [21], for NMPSO, two genetic operations (SBX and PM) are applied to the particles in the external archive. However, each new particle resulting from the genetic operations needs to be evaluated for each objective, which in turn is used by BFE to rank particles according to their weighted diversity and convergence distances [21]. Therefore, we propose a maximum amount of particles in A that can undergo genetic operations, termed Max_{GO}, to limit the number of model evaluations required for NMPSO. Max_{GO}

is expressed as a fraction of N_e, i.e., $Max_{GO} \in [0, 1]$. When $Max_{GO} \cdot N_e$ is less than N_e, the particles that undergo genetic operations are randomly selected from A.

C. Other modifications considered

In order to improve the random initialization of particles' positions described in [21], in this work we use quasi-random low-discrepancy Sobol sequences [24] to fill the decision variable space more evenly. In addition, when a particle's position, updated using (2), try to go beyond the limits of the decision variable space, the position of such particle is modified to coincide with the reached boundary, and the particle's velocity is set to zero ("absorbing2011" boundary condition in [25]).

III. Experimental setup

For this work, the NMPSO algorithm was implemented in the R environment for statistical computing and graphics [26], based on the pseudo-code and description given in [21]. The following lines briefly describe the experimental setup.

A. NMPSO configuration settings

To find a best default configuration for NMPSO in multi-objective optimization problems, three parameter settings were investigated, each one of them with four different levels: i) the swarm size used in the PSO search (N), with levels [10, 20, 30, 40]; ii) the maximum number of particles in the external archive (N_e), with levels [50, 100, 150, 200]; and iii) the maximum amount of particles in A that can undergo genetic operations (Max_{GO}, as a fraction of N_e), with levels [25%, 50%, 75%, 100%].

As the total number of all possible combinations between the four levels of N, N_e, Max_{GO} would lead to a total of 64 tests, the Taguchi design method [27] was used to reduce the number of experiments to only 16, using an orthogonal L16 arrangement, as shown in Table I.

TABLE I
NMPSO configuration settings analyzed in this work.

Set[a]	N [10,20,30,40]	N_e [50,100,150,200]	Max_{GO} [25%,50%,75%,100%]
1	10	50	25
2		100	50
3		150	75
4		200	100
5	20	50	50
6		100	25
7		150	100
7		200	75
9	30	50	75
10		100	100
11		150	25
12		200	50
13	40	50	100
14		100	75
15		150	50
16		200	25

[a]Each 3-tuple represents a different combination of N, N_e, Max_{GO}.

Following [21], the control parameters c_1 and c_2 in (1) were randomly sampled in $[1.5, 2.5]$ and the inertia weights ω was also randomly sampled in $[0.1, 0.5]$.

Regarding the genetic operators, their application involved the following specifications: i) for the SBX operator, the probabilities of doing crossover (p_c) and the distribution index (η_c) were set at 1 and 20, respectively; ii) for the PM operator, the probability of mutation (p_m) was set to $1/n$ (where n is the number of decision variables) and the distribution index (η_m) was set to 20.

B. Benchmark functions

For this work, three well-known DTLZ scalable objective problems [28] (DTLZ1, DTLZ2, DTLZ3) were used as benchmark to assess the performance of each one of the 16 NMPSO configurations defined in Section III-A and shown in Table I. These benchmark problems were implemented using the `smoof` R package [29].

Reference [28] proposed the $n = m + k - 1$ relationship to set the number of decision variables to be optimized (n) in each benchmark problem, based on the number of objective functions (m), and an integer k set to 5, 10 and 10 for the DTLZ1, DTLZ2 and DTLZ3, respectively. Therefore, considering three objective functions for each benchmark problem ($m = 3$), the number of decision variables to be optimized (n) resulted in 7, 12 and 12 for the DTLZ1, DTLZ2 and DTLZ3 problems, respectively.

C. Performance evaluation

The normalized hypervolume (HV) [30] was used in this work to simultaneously measure the spread and convergence [31] of the POF obtained with the 16 NMPSO configurations defined in Section III-A. For the three DTLZ problems, HV was computed using the volume of the search space comprised within the POF and a nadir point of reference (opposite to the ideal point towards which the POF advances), i.e., in a minimization problem each improvement of the POF should increase the corresponding HV. For the DTLZ1 function the reference point was set at $\{0.5,0.5,0.5\}$, while for DTLZ2 and DTLZ3 problems the reference point was located at $\{1,1,1\}$ [28].

To reduce the impact of the stochastic nature of NMPSO in the obtained HV, each experiment was carried out 75 times with different random seeds, where the representative value of HV at a certain number of iterations was taken as the median among all these runs.

D. Comparison with other multi-objective algorithms

The NMPSO configuration $\{N, N_e, Max_{GO}\}$ with the highest HV, out of the 16 combinations defined in Section III-A, was selected to be compared against other state-of-the-art multi-objective algorithms, specifically MMOPSO [22], NSGA-II [15] and NSGA-III [32]. These algorithms were implemented using the configuration settings defined in the `PlatEMO` Matlab platform [33], which includes a wide range of multi-objective algorithms, and also metrics and multi-objective optimization problems. All these algorithms were

configured with a population size of 100. As in NMPSO, the application of the aforementioned algorithms involves the genetic operators SBX and PM, considering the same parameters already indicated for NMPSO. Specifically for MMOPSO, the coefficients c_1 and c_2 were randomly sampled in $[1.5, 2.5]$, and the inertia weight ω was also randomly sampled, in $[0.1, 0.5]$.

To reduce the impact of the stochastic nature of the three algorithms used in this comparison, each problem was solved 75 times with each multi-objective optimization algorithm, using a different random seed for each run. The representative value of HV at a certain number of iterations was computed as the median among all these runs.

IV. Results

A. Performance of different NMPSO configuration settings

Fig. 1 shows the evolution of HV according to the number of function evaluations, for the three DTLZ problems and the 16 NMPSO configurations $\{N, N_e, Max_{GO}\}$ defined in Section III-A. In general, it is not possible to find a general behavior for all benchmark problems under all the NMPSO configurations. However, combinations with a low swarm size ($N = 10, 20$, red and violet colors) required a reduced number of function evaluations to reach a high HV value, and therefore, the following analyses will only refer to them. For all swarm sizes (i.e., all colors), the fastest convergence to the maximum HV value was reached by using 50 or 100 particles as maximum A size (square and triangle symbols), however, when $N_e = 100$ the value obtained for HV was higher than for $N_e = 50$. Finally, regarding the maximum amount of particles in A that undergo genetic operations (different line types), for DTLZ1 a 50% (dashed lines) lead to a fast convergence as well as to obtain the highest HV value, while for DTLZ2 and DTLZ3, using $Max_{GO} = 50\%$ does not always lead to the fastest convergence, but always results in the highest HV values.

B. Best configuration settings for NMPSO

Based on the results presented in Fig. 1, the combination {10,100,50%} was chosen as the best combination of $\{N, N_e, Max_{GO}\}$ to be used with NMPSO.

C. Comparison with other multi-objective algorithms

Fig. 2 shows a graphical comparison of the performance of NMPSO against MMOPSO, NSGA-II and NSGA-III. In addition, Table II shows numerical comparisons among algorithms using three measures of performance: i) the number of function evaluations required to start exceeding the POF nadir point, i.e., the $HV = 0$ line (`Evals2Start`); ii) the number of function evaluations required to achieve stabilization in the POF (`Evals2End`, i.e., when HV remains above 99% of the maximum value reached by each algorithm); and iii) HV values achieved after stabilization (HV_{max}).

These results show that the best configuration selected in this work for NMPSO ({10,100,50%}) managed to be very competitive or even superior against NSGA-II and NSGA-III

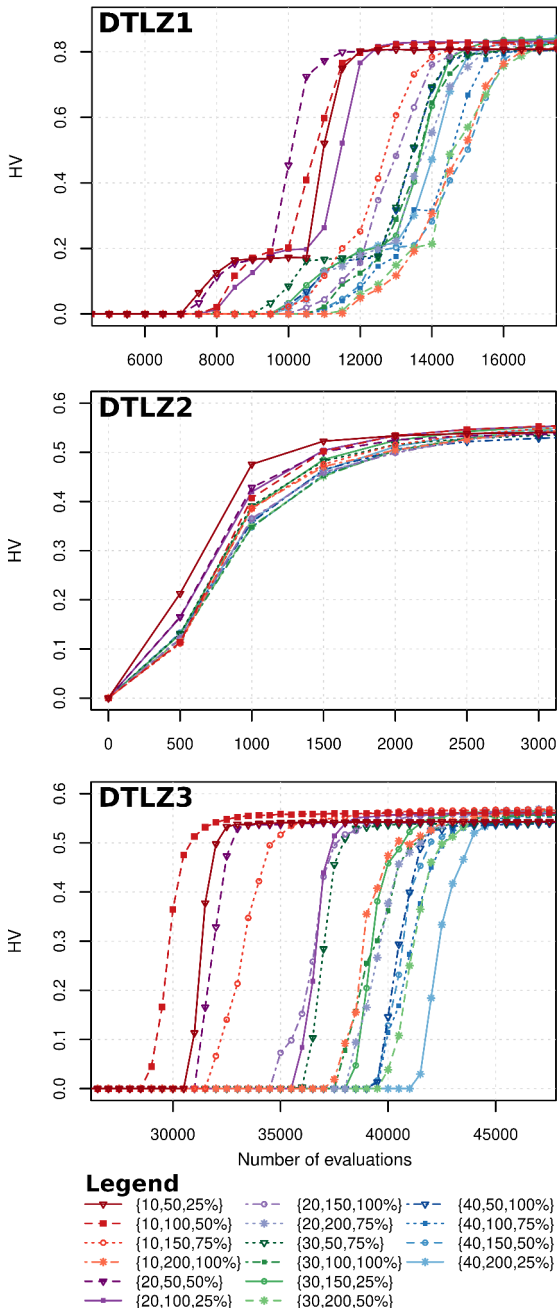

Fig. 1. Evolution of HV against the number of function evaluations in the three DTLZ benchmark problems. In the legend, colors, symbols and line types are used to group the N, N_e, and Max_{GO} values, respectively.

in terms of the three measures of performance described in the previous paragraph. In particular, for DTLZ1 the selected configuration of NMPSO is the second, after NSGA-II, to start exceeding the $HV = 0$ line (`Evals2Start=7500`), but is the fastest to stabilize (`Evals2End=14000`), and the third with the highest HV value ($HV_{max} = 0.8314$). For DTLZ2, the selected NMPSO configuration, just like the four other options tested, starts immediately to find the POF (`Evals2Start=0`), but is the fastest algorithm to converge (`Evals2End=3500`), and the one with the highest

HV ($HV_{max} = 0.5616$). Finally, for DTLZ3, the selected NMPSO configuration is the third to start looking for the POF (`Evals2Start=28500`), but again is the fastest to converge (`Evals2End=35000`) and the one that achieves the highest HV value ($HV_{max} = 0.5629$).

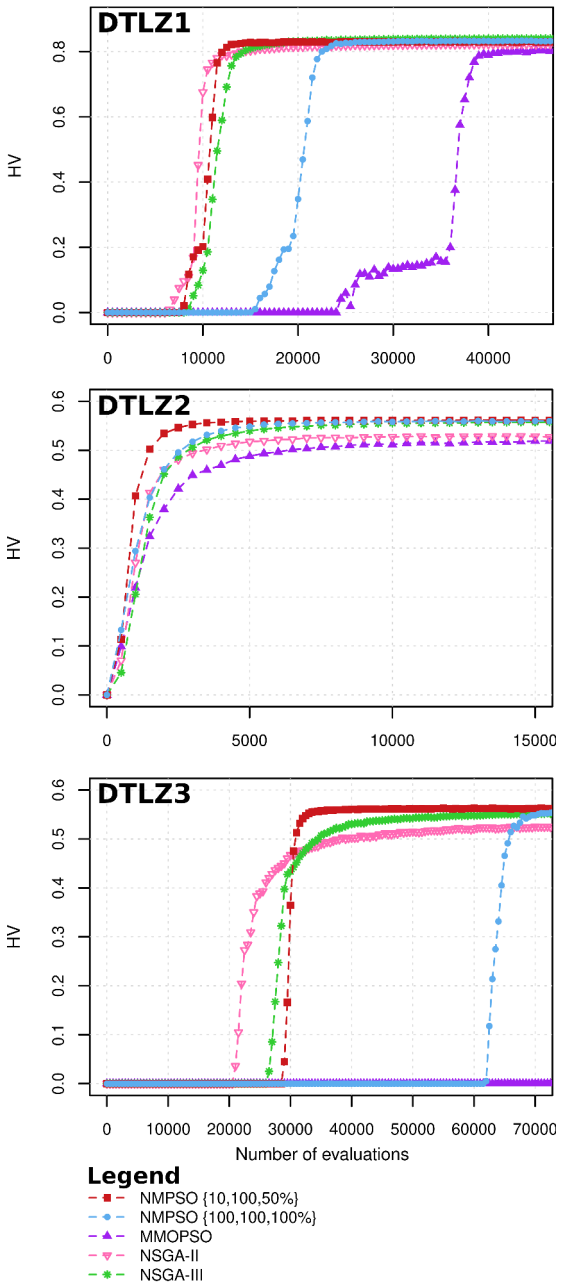

Fig. 2. Evolution of HV against the number of function evaluations for NMPSO (red squares) with configuration {10,100,50%} against MMOPSO (violet upper triangles), NSGA-II (pink down triangles), and NSGA-III (green asterisks) algorithms. In addition, the blue circle shows the performance of NMPSO with the default settings used in the `PlatEMO` Matlab platform [33].

Therefore, the selected configuration for NMPSO ($N = 10$, $N_e = 100$, $Max_{GO} = 50\%$) is competitive in providing a first approximation to the POF; however it is the algorithm with the fastest convergence to its maximum HV value; and in two out of three benchmark functions (DTLZ2, DTLZ3) is the one reaching the highest HV value (in DTLZ1 is the second best). In addition, Fig. 2 shows that the selected NMPSO configuration is way better than the default setting used in the `PlatEMO` Matlab platform [33], which is {100,100,100%}, being DTLZ3 an extreme example where the chosen NMPSO configuration reaches a stable POF at ∼28500 model evaluations while the `PlatEMO` configuration fails to do so before 70000 evaluations.

TABLE II
COMPARISON OF PERFORMANCE

		∼Evals2Start	∼Evals2End	HV_{max}
DTLZ1[a]	NMPSO{10,100,50%}	7 500	**14 000**	0.8314
	NMPSO{100,100,100%}	15 000	24 000	0.8334
	MMOPSO	24 000	41 500	0.8059
	NSGA-II	**6 000**	21 000	0.8236
	NSGA-III	8 000	21 000	**0.8410**
DTLZ2[a]	NMPSO{10,100,50%}	**0**	**3 500**	**0.5616**
	NMPSO{100,100,100%}	**0**	7 000	0.5606
	MMOPSO	**0**	11 000	0.5196
	NSGA-II	**0**	7 000	0.5302
	NSGA-III	**0**	8 500	0.5584
DTLZ3[a]	NMPSO{10,100,50%}	28 500	**35 000**	**0.5629**
	NMPSO{100,100,100%}	61 500	$\not<$70 000	0.5612
	MMOPSO	-	$\not<$70 000	-
	NSGA-II	**20 500**	57 000	0.5260
	NSGA-III	26 000	57 500	0.5533

[a]The maximum number of evaluations made with the DTLZ1, DTLZ2 and DTLZ3 tests were 50000, 20000 and 75000, respectively.

V. CONCLUSIONS

In this work we tested different configuration settings for NMPSO, aiming at finding an optimal configuration that improves its performance in multi-objective optimization problems, focusing on obtaining optimal results with the fewest possible number of function evaluations. In particular, we used an orthogonal L16 arrangement to select 16 different combinations of: i) the swarm size (N), ii) the maximum number of particles in the external archive (N_e), and iii) the maximum amount of genetic operations allowed for the external archive (Max_{GO}). Three DTLZ problems were used, with three objective functions each, to test the previous configuration settings. The best configuration was then compared against other MOPs (MMOPSO, NSGA-II, NSGA-III), and against NMPSO with the default configuration settings used in the well-known `PlatEMO` Matlab platform.

Our results showed that configurations with a low amount of particles in the swarm ($N = 10, 20$) reached the maximum hypervolume (HV) earlier than configurations with a larger amount of particles ($N = 30, 40$), which indicates that the velocity update equation used in the PSO search has still space for improvement. Configurations with a low amount of particles in the swarm and 50-100 particles as maximum size of the external archive showed the fastest convergence to the maximum HV value; and using $Max_{GO} = 50\%$ does not always lead to the fastest convergence, but always resulted in the highest HV values. Therefore, the combination $\{N = 10, N_e = 100, Max_{GO} = 50\%\}$ was chosen as the best

configuration to be used with NMPSO to solve the selected MOPs.

Our results showed that the selected NMPSO configuration is very competitive or even superior against NSGA-II and NSGA-III in terms of the number of function evaluations required to start having an HV larger than zero, and to achieve stabilization in the POF; but also in the maximum HV values achieved after stabilization. In addition, our results showed that the selected NMPSO configuration was much better than the default settings used in PlatEMO.

In this way, this work contributes to define an optimal configuration for the application of NMPSO in real-world multi-objective optimization problems, where the number of function evaluations is an important constraint to be addressed due to the long execution times of the functions to be optimized (e.g., numerical models).

ACKNOWLEDGMENTS

This work was partially funded by ANID PCI NSFC 190018 (*Management of global change impacts on hydrological extremes by coupling remote sensing data and an interdisciplinary modeling approach*) and ANID Fondecyt 1212071 (*The catchment's memory: understanding how hydrological extremes are modulated by antecedent soil moisture conditions in a warmer climate*).

REFERENCES

[1] P. Reed, D. Hadka, J. Herman, J. Kasprzyk, and J. Kollat, "Evolutionary multiobjective optimization in water resources: The past, present, and future," *Advances in Water Resources*, vol. 51, pp. 438–456, 2013.

[2] G. Chiandussi, M. Codegone, S. Ferrero, and F. Varesio, "Comparison of multi-objective optimization methodologies for engineering applications," *Computers & Mathematics with Applications*, vol. 63, no. 5, pp. 912–942, 2012.

[3] K. Metaxiotis and K. Liagkouras, "Multiobjective evolutionary algorithms for portfolio management: A comprehensive literature review," *Expert Systems with Applications*, vol. 39, no. 14, pp. 11 685–11 698, 2012.

[4] A. Alarcon-Rodriguez, G. Ault, and S. Galloway, "Multi-objective planning of distributed energy resources: A review of the state-of-the-art," *Renewable and Sustainable Energy Reviews*, vol. 14, no. 5, pp. 1353–1366, 2010.

[5] C. A. C. Coello, "Evolutionary multi-objective optimization: some current research trends and topics that remain to be explored," *Frontiers of Computer Science in China*, vol. 3, no. 1, pp. 18–30, 2009.

[6] R. Marler and J. Arora, "Survey of multi-objective optimization methods for engineering," *Structural and Multidisciplinary Optimization*, vol. 26, no. 6, p. 369, 2004.

[7] C. A. C. Coello, "Evolutionary Multi-Objective Optimization: A Critical Review," *Evolutionary Optimization*, pp. 117–146, feb 2003.

[8] K. Deb, "Multi-objective Optimisation Using Evolutionary Algorithms: An Introduction," *Multi-objective Evolutionary Optimisation for Product Design and Manufacturing*, pp. 3–34, 2011.

[9] S. Cheng, Y. Shi, and Q. Qin, "On the Performance Metrics of Multi-objective Optimization," *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 7331 LNCS, no. PART 1, pp. 504–512, 2012.

[10] M. Farina and P. Amato, "On the optimal solution definition for many-criteria optimization problems," *Annual Conference of the North American Fuzzy Information Processing Society - NAFIPS*, vol. 2002-January, pp. 233–238, 2002.

[11] N. Gunantara, "A review of multi-objective optimization: Methods and its applications," *Cogent Engineering*, vol. 5, no. 1, pp. 1–16, jan 2018.

[12] C. A. Coello Coello and M. S. Lechuga, "MOPSO: A proposal for multiple objective particle swarm optimization," *Proceedings of the 2002 Congress on Evolutionary Computation, CEC 2002*, vol. 2, pp. 1051–1056, 2002.

[13] C. A. Coello Coello, G. T. Pulido, and M. S. Lechuga, "Handling multiple objectives with particle swarm optimization," *IEEE Transactions on Evolutionary Computation*, vol. 8, no. 3, pp. 256–279, jun 2004.

[14] K. Deb, "Multi-objective evolutionary algorithms," in *Springer handbook of computational intelligence*. Springer, 2015, pp. 995–1015.

[15] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, apr 2002.

[16] Q. Zhang and H. Li, "MOEA/D: A multiobjective evolutionary algorithm based on decomposition," *IEEE Transactions on Evolutionary Computation*, vol. 11, no. 6, pp. 712–731, dec 2007.

[17] M. R. Sierra and C. A. C. Coello, "Improving PSO-Based Multi-objective Optimization Using Crowding, Mutation and e-Dominance," *Lecture Notes in Computer Science*, vol. 3410, pp. 505–519, 2005.

[18] A. J. Nebro, J. J. Durillo, G. Nieto, C. A. Coello, F. Luna, and E. Alba, "SMPSO: A new pso-based metaheuristic for multi-objective optimization," *2009 IEEE Symposium on Computational Intelligence in Multi-Criteria Decision-Making, MCDM 2009 - Proceedings*, pp. 66–73, 2009.

[19] Z. H. Zhan, J. Li, J. Cao, J. Zhang, H. S. H. Chung, and Y. H. Shi, "Multiple populations for multiple objectives: A coevolutionary technique for solving multiobjective optimization problems," *IEEE Transactions on Cybernetics*, vol. 43, no. 2, pp. 445–463, apr 2013.

[20] W. Hu and G. G. Yen, "Adaptive multiobjective particle swarm optimization based on parallel cell coordinate system," *IEEE Transactions on Evolutionary Computation*, vol. 19, no. 1, pp. 1–18, feb 2015.

[21] Q. Lin, S. Liu, Q. Zhu, C. Tang, R. Song, J. Chen, C. A. C. Coello, K.-C. Wong, and J. Zhang, "Particle swarm optimization with a balanceable fitness estimation for many-objective optimization problems," *IEEE Transactions on Evolutionary Computation*, vol. 22, no. 1, pp. 32–46, 2018.

[22] Q. Lin, J. Li, Z. Du, J. Chen, and Z. Ming, "A novel multi-objective particle swarm optimization with multiple search strategies," *European Journal of Operational Research*, vol. 247, no. 3, pp. 732–744, dec 2015.

[23] J. Kennedy and R. Eberhart, "Particle swarm optimization," *Proceedings of ICNN'95 - International Conference on Neural Networks*, vol. 4, pp. 1942–1948, 1995.

[24] I. M. Sobol′, "On the distribution of points in a cube and the approximate evaluation of integrals," *USSR Computational Mathematics and Mathematical Physics*, vol. 7, no. 4, pp. 86–112, 1967.

[25] M. Zambrano-Bigiarini and R. Rojas, "A model-independent Particle Swarm Optimisation software for model calibration," *Environmental Modelling & Software*, vol. 43, pp. 5–25, may 2013.

[26] R Core Team, *R: A Language and Environment for Statistical Computing*, R Foundation for Statistical Computing, Vienna, Austria, 2021. [Online]. Available: https://www.R-project.org/

[27] G. Taguchi, *Introduction to quality engineering: designing quality into products and processes*. Asian Productivity Organization, 1986.

[28] K. Deb, L. Thiele, M. Laumanns, and E. Zitzler, *Scalable Test Problems for Evolutionary Multiobjective Optimization*. London: Springer London, 2005, pp. 105–145.

[29] J. Bossek, "smoof: Single- and multi-objective optimization test functions," *The R Journal*, 2017. [Online]. Available: https://journal.r-project.org/archive/2017/RJ-2017-004/index.html

[30] E. Zitzler and L. Thiele, "Multiobjective evolutionary algorithms: a comparative case study and the strength pareto approach," *IEEE Transactions on Evolutionary Computation*, vol. 3, no. 4, pp. 257–271, 1999.

[31] N. Riquelme, C. Von Lucken, and B. Baran, "Performance metrics in multi-objective optimization," in *2015 Latin American Computing Conference (CLEI)*, 2015, p. 1.

[32] K. Deb and H. Jain, "An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, Part I: Solving problems with box constraints," *IEEE Transactions on Evolutionary Computation*, vol. 18, no. 4, pp. 577–601, 2014.

[33] Y. Tian, R. Cheng, X. Zhang, and Y. Jin, "PlatEMO: A MATLAB platform for evolutionary multi-objective optimization," *IEEE Computational Intelligence Magazine*, vol. 12, no. 4, pp. 73–87, 2017.